# Namal University, Mianwali

## Department of Computer Science

# Machine Learning

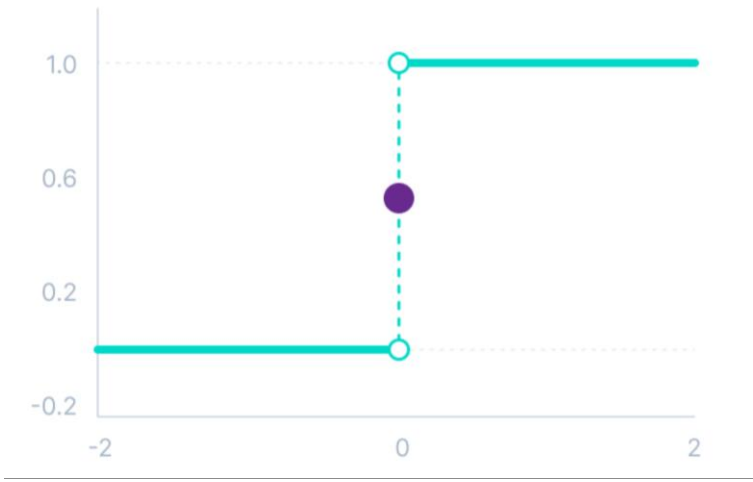
# Activation Functions

# Activations

- Step Function
- Linear Function
- Rectified Linear Unit (ReLU)
- Leaky ReLU
- Sigmoid
- TanH
- Softmax

# Binary Step Function

- Binary step function depends on a threshold value that decides whether a neuron should be activated or not.
- Limitations:
- It cannot provide multi-value outputs—for example, it cannot be used for multi-class classification problems.
- The gradient of the step function is zero, which causes a hindrance in the backpropagation process.




$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$

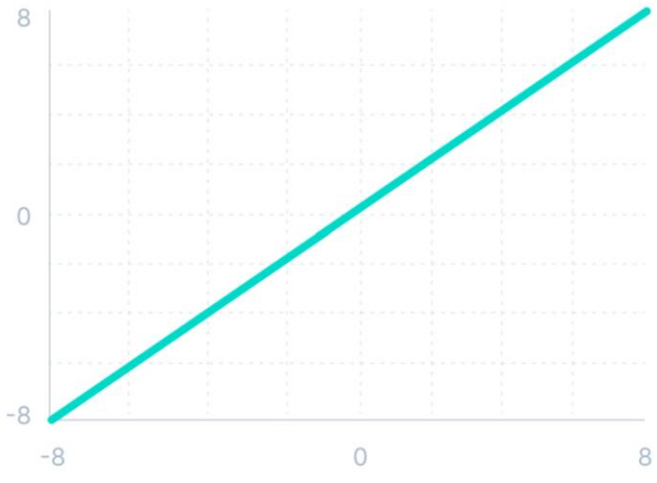
# Linear Activation Function

The linear activation function is also known as "no activation," or "identity function".

All layers of the neural network will collapse into one if a linear activation function is used.

No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer.

So, essentially, a linear activation function turns the neural network into just one layer.




$$f(x) = x$$

# Non-Linear Activation Functions

Non-linear activation functions solve the following limitations of linear activation function:

They allow backpropagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.

They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers.

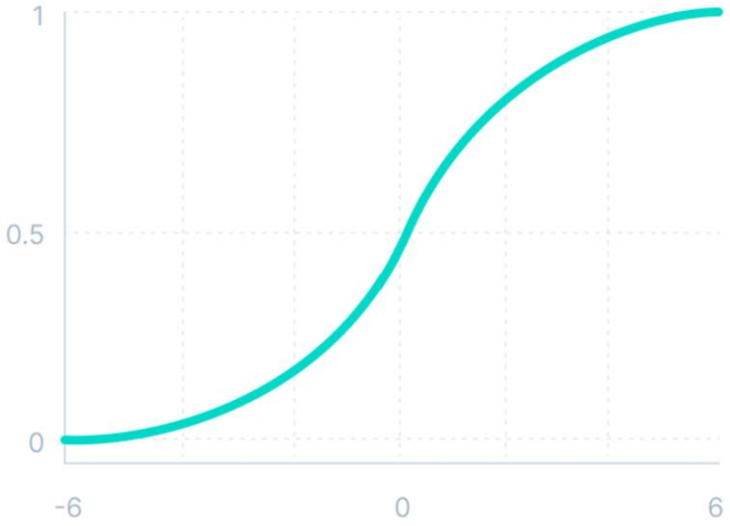
# Sigmoid (Logistic) Activation Function

This function takes any real value as input and outputs values in the range of 0 to 1.

The larger the input, the closer the output value will be to 1, whereas the smaller the input, the closer the output will be to 0.

It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.

The function is differentiable and provides a smooth gradient.




$$f(x) = \frac{1}{1 + e^{-x}}$$

# Sigmoid (Logistic) Activation Function

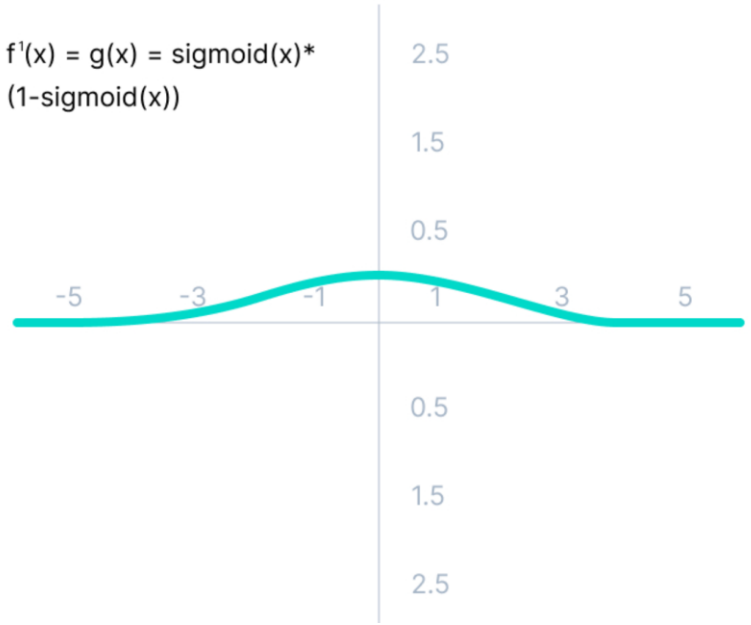
The limitations of sigmoid function are discussed below:

The derivative of the function is

$$f'(x) = sigmoid(x)*(1-sigmoid(x))$$

The gradient is only significant for range -3 to 3, and the graph gets much flatter in other regions.

As the gradient value approaches zero, the network ceases to learn and suffers from the *Vanishing gradient* problem.



# Hyperbolic Tangent (Tanh) Activation

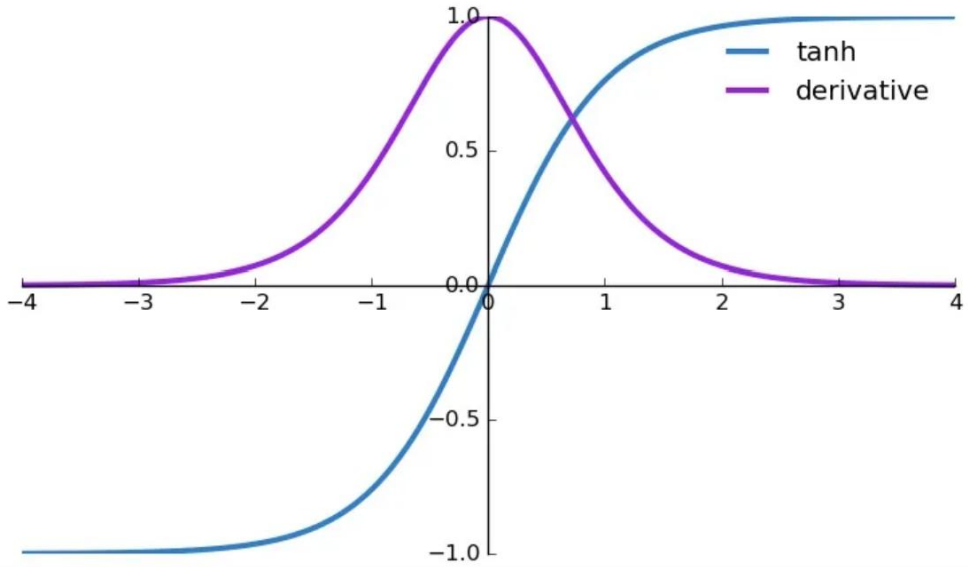
The output of the tanh activation function is Zero centered; hence we can easily map the output values as strongly negative, neutral, or strongly positive.

As its values lie between -1 to 1, it helps in centering the data and makes learning for the next layer easier.

It also faces the problem of vanishing gradients like the sigmoid activation function.

$$a = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$\frac{da}{dz} = 1 - a^{2}$$



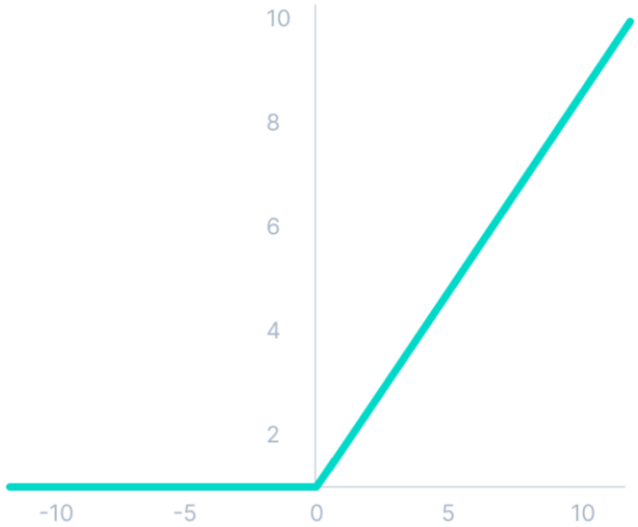
# Rectified Linear Unit - ReLU

Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation.

The main catch is that the ReLU function does not activate all the neurons.

Since only a certain number of neurons are activated, it is more computationally efficient when compared to the sigmoid and tanh functions.

ReLU does not suffer from the vanishing gradient problem.

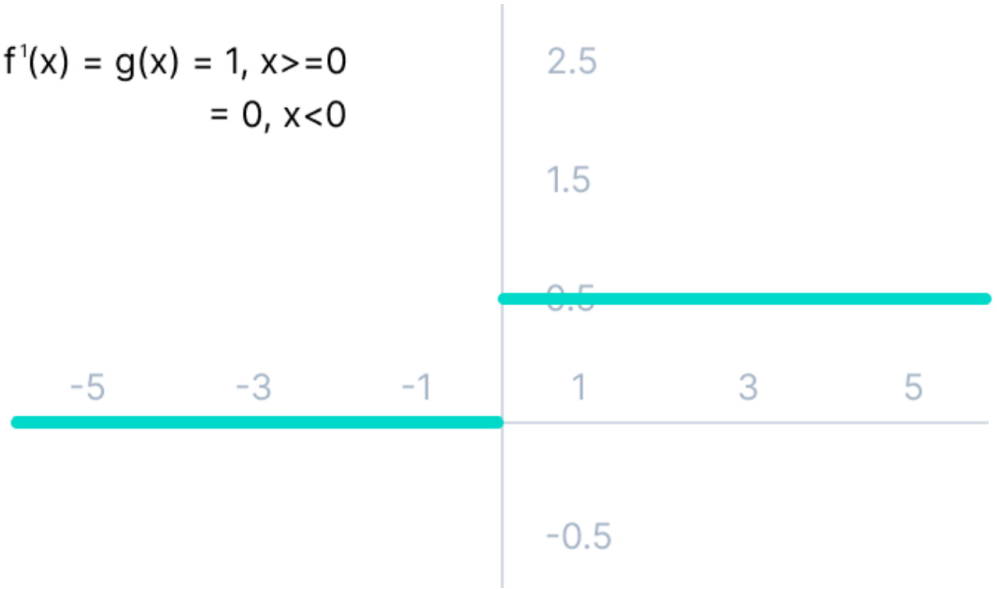



$$f(x) = max\ (0, x)$$

# Rectified Linear Unit - ReLU

The issue with this activation is the Dying ReLU problem.

The negative side of the graph makes the gradient value zero. Due to this reason, during the backpropagation process, the weights and biases for some neurons are not updated. This can create dead neurons which never get activated.



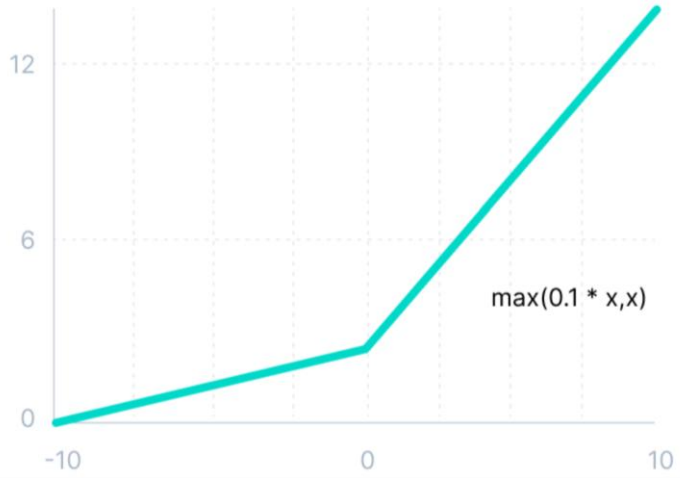
# Leaky ReLU

Improved version of ReLU to solve the Dying ReLU problem as it has a small slope in the negative side.

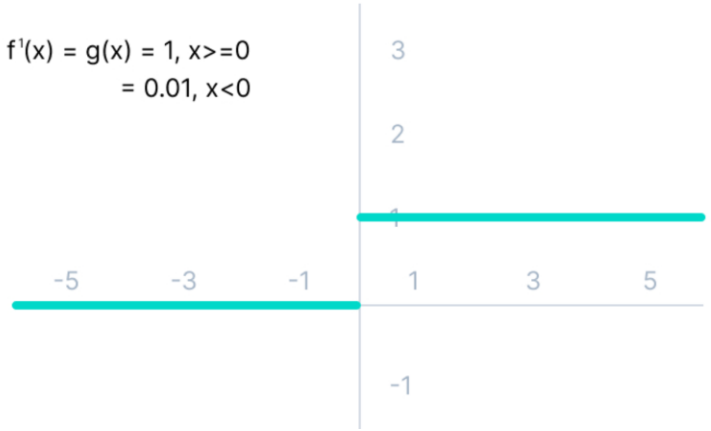
By making this minor modification for negative input values, the gradient of the left side of the graph comes out to be a non-zero value. Therefore, we would no longer encounter dead neurons in that region.

The advantages of Leaky ReLU are same as that of ReLU, in addition to the fact that it does enable backpropagation, even for negative input values.

The gradient for negative values is a small value that makes the learning of model parameters time-consuming.




$$f(x) = max\,(0.1x, x)$$



# Softmax Activation Function

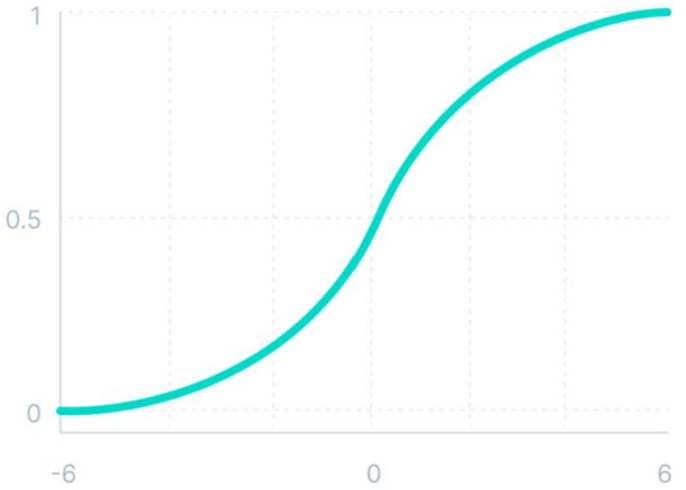
It is most commonly used as an activation function for the last layer of the neural network in the case of multi-class classification.

Assume that you have three neurons in the output layer with output [1.8, 0.9, 0.68].

Applying the Softmax function over these values will result in [0.58, 0.23, 0.19].

The function returns 1 for the largest probability index while it returns 0 for the other two array indexes.

You can see now how Softmax activation function make things easy for multi-class classification problems.




$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Thank you